

DATE: _____

DATE: _____

EAD-01

(20/01/25)

⇒ Framework

↳ provide facility.

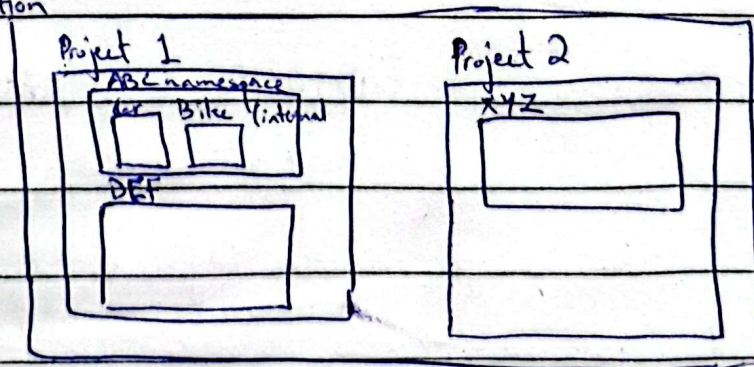
↳ Already written code.

EAD-02

(23/01/25)

dll files → own library files/packages.

Solution



⇒ One sol can have d/f projects.

1) Desktop

2) Mobile

Class Member Access Modifiers:

- 1) Public → Other
- 2) Private →
- 3) Internal →

Ex → 1) 2 project link.

2) External library link

3) Nuget package.

// no of parameters in func not fixed.

```
public int sum(params int [] elements)
```

```
{ int sum = 0;
```

```
    foreach (int ele in elements)
```

```
        sum += ele;
```

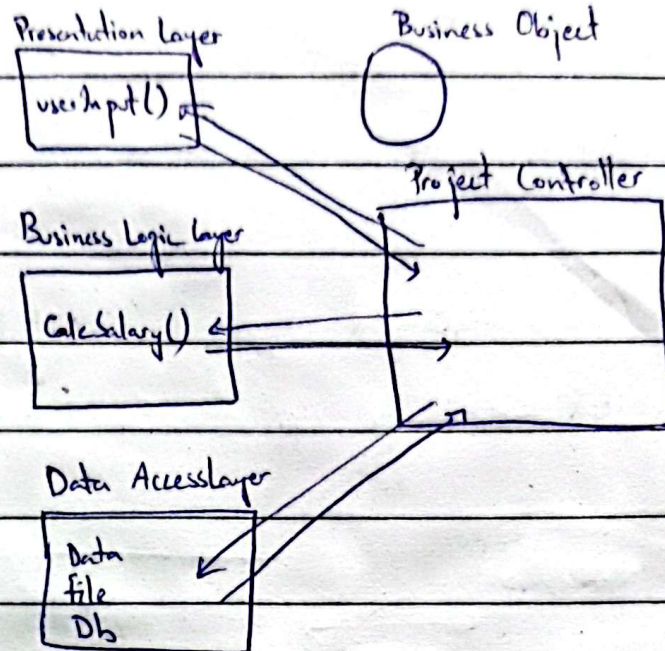
```
    return sum;
```

```
}
```

→ System + some other namespaces included by default.

EAD-04

(30/01/25)

N-tier Architecture:

⇒ If Db needs to be changed only
Data Access Layer will be changed.

⇒ Desktop to Mobile ⇒ Presentation Layer.

- 1) Add Project (Library - Presentation Layer)
- 2) _____ Business Logic Layer)
- 3) _____ Data Access Layer)
- 4) _____ Business Object Layer)

⇒ Add project ref & namespaces to
main Controller & then use classes.

⇒ getter/setter ⇒ imposing logic like `age > 10`

⇒ `Emp emp1 = new Emp {Age = 15, ...}`


```

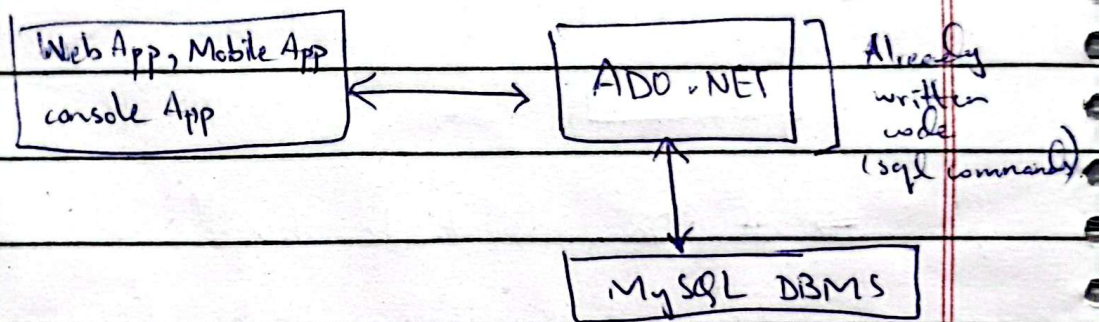
⇒ private string str-name;

public string Str-name {
    get { return str-name; }
    set { str-name = value; }
}
    
```

EAD-05

(03/02/25)

ADO.NET → connects Application & Data Source.



CRUD

SQL Injection
 → Parameterized Query (Solution)
 → Entity Framework.

⇒ Microsoft SQL Client Package.

⇒ SQL Server Object Explorer.

⇒ IDENTITY PRIMARY KEY.

↳ Auto key + increment.

⇒ Select Query → ExecuteReader()

⇒ Insert, Update, Del → ExecuteNonQuery()

-- for commenting queries.

EAD-06 (06/02/25)

→ Sol of SQL Injection.

↳ Parameterized queries:

~ where username = '@abc' ~

```
SqlParameter p1 = new SqlParameter("abc", username);
```

```
cmd.Parameters.Add(p1);
```

1- Create connection (conn string).

2- Create command (query, conn)

3- Add Parameters to command

4- Execute query.

⇒ use try catch.

// some objects need to be deleted from RAM

ASAP

```
↳ using (Program p = new Program())
    { //use here }
```

↳ require IDisposable interface to be implemented.

```
class car() : IDisposable {
    public void Dispose() {
        ~~~~~
    }
}
```

EAD-07

(17/02/25)

⇒ WPF Application Project

⇒ backend (C#), frontend (XML)

↳ Drag Drop (not responsive)
↳ code

⇒ 4 Panels for frontend:

1) dock 2) stack 3) wrap 4) grid.

Stack Panel: (single row / col)

↳ Orientation ⇒ default vertical.

↳ nested ~~stack~~ panels possibleDock Panel: ~~Example~~↳ When we want to dock
element e.g. top

↳ dock = "left" (default).

↳ lastChildFill = "True"

↳ takes remaining space.

Grid Panel

↳ Divides the UI into rows & cols.

↳ Wrap Panel:

↳ Similar to StackPanel, but wraps items to the next row/column when there's no space.

EAD-08

(19/02/25)

NuGet Package Tools ⇒ (Package Manager Installation)

1) Microsoft . Entity FrameworkCore . Sql Server

2) _____ . Design

3) _____ . Tools

↳ Tables should already exist in db.

↳ makes class from table.

(Table) users ⇒ (Class) User

↳ DbSet <User> users;

⇒ Package Manager Console.

↳ // for db connection.

SqlConnection db = new SqlConnection();

List<User> list = db.Users.ToList();

DbSet(table) ↴

```
using (SqlConnection db = new SqlConnection()) {
    VetUser tmp = new VetUser { username = "VET",
                                password = "KSK123" };
    db.VetUsers.Add(tmp);
    db.SaveChanges();
}
```

EAD-09

(24/02/25)

↳ Class first Approach

↳ DB first Approach

⇒ Entity framework makes classes from tables & list of objects in class of DB-NAME.

```
SqlConnection db = new SqlConnection();
User obj; //only ref var is made.
```

// get list

```
List<VetUser> lst = db.VetUsers.ToList();
```

// insert obj

```
User tmp = new User {
```

```
    db.VetUser.Add(tmp);
```

```
    db.SaveChanges();
```

↳ To sync RAM changes with DB.

DATE: _____

DATE: _____

```
// delete obj; var obj = db.VetUsers.FirstOrDefault  
                (x => x.username == "UET")
```

```
db.VetUsers.Remove(obj);
```

```
db.SaveChanges();
```

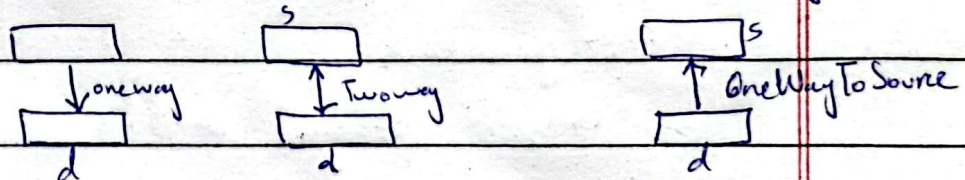
```
// update obj;
```

```
// get // edit // saveChanges();
```

Imp Data Binding:

1) Assign names 2) Remove text property.

3) Text = " { Binding ElementName = source
 Path = Text Mode = OneWay }



→ Two way mode require

UpdateSourceTriggers = PropertyChanged.

(to update automatically).

Example:

Text Box ☐ checkbox.

without binding { if (textbox.text == "check")
 Check1.IsChecked = true;

DATE _____

DATE _____

↳ make Converter class.

TextToBool.

< Window.Resources >

<local:TextToBool x:key="myConverter" >

</local:TextToBool >

</ Window.Resources >

⇒ is Checked = " { Binding ElementName= Ty-source
Path=Text

Converter = { Static Resource myConverter
} }

Exercise:

↳ convert back.

↳ bind datagrid (edit/delete).

C

DATE _____

DATE _____

EAD-10

(27/02/25)

List<Product> lst = new List<Product>();

↳ List contains list of addresses.

DataGrid
Binding

⇒ mygrid.

<DataGrid Name="mygrid" AutoGenerateColumns="false"
canUserAddRows="false">

<DataGrid.Columns>

<DataGridTextColumn Header=" " width="40"

Binding="{Binding path}"

</Data

<DataGridTemplateColumn

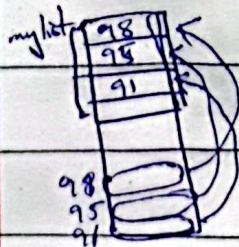
↳ frontend will not auto update.

list ⇒ ObservableCollection.

list does not
handle notify

// Product tmp = (mygrid.SelectedRow as Product)

lst.remove(tmp).



mylist ⇒ Link list.

DATE _____

DATE _____

EAD-11

(3/3/25)

```
try { catch (Exception e) { }
```

↳ will avoid app to crash.

// Edit/Update

⇒ when edit ^(any) button is clicked in grid
it becomes selected.

⇒ We can get address of selected row as
Product temp = (mygrid.getSelectedItem as Product);

⇒ When obj is added/deleted in list,
it notifies grid but when
existing obj is changed it
does not notifies ~~the~~ list

```
⇒ class Product: INotifyPropertyChanged {  
    private int price;  
    public int Price { get { return price; }  
        set { price = value;  
            Notify("Price");  
        }  
    }  
}
```